CodeCraft

Intelligent Code Auto Formatter
Project Presentation Guide

What is the project about?

CodeCraft takes poorly formatted C code, checks its syntax, and produces readable code with consistent indentation, spacing, and brace placement. It also displays tokens and an abstract syntax tree so users can understand how their code was analyzed.

> Our project applies compiler construction concepts to code formatting. It performs lexical analysis, syntax analysis, and AST construction, then uses that information to format C code while preserving its original tokens and structure.

Technologies used

Technology	Purpose
Python	Compiler and formatting logic
PLY (Python Lex-Yacc)	Lexer and LALR(1) parser
Flask and Flask-CORS	REST API and local cross-origin access
React and Vite	Interactive interface, development server, production build
Tailwind CSS	Interface styling
Monaco Editor	Source editor and code comparison
React Flow	Interactive AST visualization
Axios	Frontend-to-backend HTTP requests
Framer Motion and Lucide	Animations and icons
pytest and Playwright	Backend and browser workflow tests

How we implemented it

The application has a React frontend and a Python Flask backend.

```
User enters C code
        ↓
React sends it to the Flask API
        ↓
Lexer converts source into tokens
        ↓
Parser checks syntax and constructs an AST
        ↓
Formatter applies the selected layout
        ↓
Backend verifies the formatted output
        ↓
Frontend displays results and analysis
```

1. Source input and settings

The user types or uploads C code in Monaco Editor. Settings include two spaces, four spaces, or tabs; brace placement; and blank lines between functions.

2. Lexical analysis

In backend/compiler/lexer.py, PLY Lex recognizes keywords, identifiers, numbers, strings, operators, punctuation, comments, and supported include directives. It records source positions for token tables and diagnostics.

For example, `int x = 10;` becomes the tokens: keyword `int`, identifier `x`, assignment operator `=`, number `10`, and delimiter `;`.

3. Syntax analysis and AST construction

In backend/compiler/parser.py, grammar rules written with PLY Yacc produce an LALR(1) parser. The parser checks whether tokens follow the supported C grammar and constructs an Abstract Syntax Tree (AST), a structured representation of the program.

```
return x + 2;

ReturnStatement
└── BinaryExpression (+)
    ├── Identifier: x
    └── Literal: 2
```

Invalid input produces diagnostics with a line, column, message, and suggestion.

Formatting and verification

4. Formatting

In backend/compiler/formatter.py, the formatter uses the AST to identify nesting, statement boundaries, unary operators, and function boundaries. It renders the original token text with consistent spacing, indentation, and line breaks. It preserves comments and literal contents and rejects invalid input before formatting.

Input:

```
int main(){int x=1;if(x>0){x+=2;}return x;}
```

Output:

```
int main() {
    int x = 1;
    if (x > 0) {
        x += 2;
    }
    return x;
}
```

5. Verification

The backend lexes and parses the formatted output again. It requires the same ordered non-whitespace token sequence, including comments and directives, and the same AST after removing source-position metadata.

These checks verify token and structural preservation within the supported subset. They do not prove full C semantic equivalence.

6. Displaying the results

The API returns formatted code, tokens, AST, diagnostics, and statistics. React presents an output editor, searchable token analyzer, interactive syntax tree, side-by-side or inline comparison, and statistics. Users can copy or download the output.

Frontend and backend communication

Vite runs on port 5173 and Flask on port 5000. During development, Vite forwards /api requests to Flask. The main endpoint is POST /api/format. Other endpoints provide tokenization, parsing, analysis, and a health check.

```
{
  "source": "int main(){return 0;}",
  "options": {
    "indent": "4",
    "braceStyle": "same-line",
    "blankLines": true
  }
}
```

Scope and limitations

CodeCraft is a compiler front-end application for a documented subset of C. It does not execute source code or generate machine code.

It supports basic declarations, functions, expressions, conditions, loops, arrays, and return statements. Pointer declarations, structs, full macro preprocessing, switch statements, and other advanced C features are outside the current scope. It does not perform full semantic analysis or type checking.

Short presentation script

> Our project, CodeCraft, is an intelligent formatter for a subset of C. Its purpose is to improve code readability while demonstrating compiler construction concepts.
>
> We built the frontend using React and the backend using Python and Flask. For lexical and syntax analysis, we used PLY, which provides Lex and Yacc functionality.
>
> When a user submits code, the lexer produces tokens. The parser validates those tokens against our grammar and constructs an abstract syntax tree. Our formatter uses this structure to apply consistent indentation and spacing.
>
> We then tokenize and parse the formatted output again to verify that its tokens and syntax tree remain unchanged. The application also provides a token analyzer, syntax tree visualization, code comparison, and error diagnostics.
>
> The main learning outcome is implementing and connecting the compiler front-end phases in an interactive application.

Suggested demonstration

1. Format the preloaded sample code.
2. Show the token analyzer and explain a few tokens.
3. Open the syntax tree and describe its structure.
4. Change indentation or brace placement and format again.
5. Show the original and formatted code comparison.
6. Submit invalid code to demonstrate error reporting.

Commands to run the project

From the project root, use two PowerShell terminals.

```
# Terminal 1: backend
.\.venv\Scripts\python.exe backend\app.py

# Terminal 2: frontend
cd frontend
npm run dev
```

Open http://127.0.0.1:5173 and keep both terminals running.